

InternTrack — Deployment Handover

Prepared for **Carlo** · 28 July 2026 · repository at commit `ccec6e9` · branches `main` and `deploy` both pushed

Start here

This sheet is orientation only. The real step-by-step guide is [docs/DEPLOYMENT.md](#) in the repo — 12 sections covering every environment variable, the exact click-path per service, a verification script, and a symptom→cause troubleshooting table. Read that when you sit down to deploy. Everything below is the short version plus the things easiest to get wrong.

1. What you are deploying

Layer	Service	Cost
Vue SPA (<code>web/</code>)	Vercel Hobby	Free, no card
Laravel API (repo root)	Render free web service, via Docker	Free, no card
Database	Aiven managed MySQL, free plan	Free, no card, does not expire

Two deliberate choices — please do not undo them

- **The database is Aiven, not Render's own.** Render's free PostgreSQL **expires 30 days after creation**, and this app is built on MySQL. Switching database engines shortly before a defense is risk with no upside.
- **The SPA reaches the API through Vercel rewrites, not cross-domain calls.** That keeps the Laravel session cookie first-party. The alternative requires `SameSite=None`, which makes login depend on third-party cookies that Safari and hardened browsers block outright.

2. The three things most likely to trip you up

Trap	What to do
Vercel Root Directory	Set it to <code>web</code> . The repo root is a Laravel app — pointing Vercel there fails or deploys nothing useful. This is the most common mistake.
7 placeholders in <code>web/vercel.json</code>	After Render assigns a URL, replace every <code>REPLACE-WITH-API-HOST</code> with your Render hostname (no <code>https://</code> , no trailing slash), then commit. These cannot be environment variables — Vercel does not interpolate env vars inside rewrite destinations.
Vercel Production Branch	Set it to <code>deploy</code> in Settings → Git. There is no <code>vercel.json</code> field for this, and <code>main</code> is deliberately disabled — so without this step Vercel builds nothing at all .

3. Your steps, condensed

Full detail in [docs/DEPLOYMENT.md](#) sections 3–7. Note that Render and Vercel each need the other's URL, so you pass through Render's variables twice.

1. **Approve the Render and Vercel GitHub Apps** on the repo. You own it, so this is yours to grant.
2. **Collect three values first:** `APP_KEY` (run `php artisan key:generate --show`), a `DEMO_PASSWORD` (12+ characters, *not* password), and a Vercel project name.
3. **Aiven** → free MySQL service → copy host, port, database, user, password.
4. **Render** → New → Blueprint → the repo. It reads `render.yaml` and configures itself. Fill in `APP_KEY`, the five `DB_*` values, `SEED_ON_BOOT=demo` and `DEMO_PASSWORD`. Deploy, then confirm `curl https://YOUR-API.onrender.com/up` returns **200** before going further.
5. **Fill the 7 placeholders** in `web/vercel.json` and commit.
6. **Vercel** → import the repo → Root Directory `web` → Production Branch `deploy`. Deploy, then copy the assigned domain.
7. **Back to Render** and set `APP_URL`, `FRONTEND_URL`, `SANCTUM_STATEFUL_DOMAINS` (the Vercel *host only*, no scheme) and `GOOGLE_REDIRECT_URI`. Redeploy — then **remove** `SEED_ON_BOOT`.
8. **Google Cloud** (optional, sign-in works without it): add the Vercel callback URL, and add each panelist's Gmail under Test users.

4. Where the documentation lives

All of these are committed and pushed on both `main` and `deploy`.

File	What it is for
<code>docs/DEPLOYMENT.md</code>	The runbook — read this one. Prerequisites, per-service steps, every environment variable with the reason it must be that exact value, a post-deploy verification script, a troubleshooting table, and a "never do these" list.
<code>.env.production.example</code>	Every environment variable the API takes, with the reasoning inline. No real values, so it is safe to read and safe to commit.
<code>render.yaml</code>	The API service definition Render reads: free plan, Singapore region, Docker build, <code>/up</code> health check, and the <code>deploy</code> branch pin. Secrets are marked so Render prompts instead of committing them.
<code>web/vercel.json</code>	SPA build settings plus the proxy rewrites. This is the file with the 7 placeholders.
<code>Dockerfile</code> <code>docker/entrypoint.sh</code>	How the API image is built and what happens on every boot: port binding, migrations, optional seeding, cache warming.
<code>CLAUDE.md</code>	Architecture, domain rules, and the full deployment decision record — search for DECIDED STACK . Explains <i>why</i> each choice was made.
<code>README.md</code>	Project overview, local development setup, demo accounts.
<code>docs/PROJECT_HISTORY.md</code>	How the project got here, including what changed for deployment.

5. Never do these

Each of these has a real consequence

- **Never run bare** `php artisan db:seed` on the deployment. It calls twelve *demo* seeders and creates roughly 30 fictional accounts whose shared password is documented in this **public** repository — including an admin. Use `SEED_ON_BOOT=demo` (which rotates them all immediately) or `db:seed --class=ProductionSeeder`.
- **Never set** `APP_DEBUG=true` in production. The app refuses to boot in that state by design, because it would leak stack traces publicly.
- **Never commit a filled-in** `.env`. It is gitignored; the host supplies real environment variables.
- **Never leave** `SEED_ON_BOOT` **set** after the first successful boot. The service re-runs its entrypoint every time it wakes from sleep, so a stale value could re-seed over live data.

- **Never point `SANCTUM_STATEFUL_DOMAINS` at the API's own domain.** It must be the Vercel host. Get this wrong and login *appears* to succeed while every authenticated request returns 401.

6. Shipping changes after it is live

Deploys track the `deploy` branch, **not** `main`. Pushing to `main` has no effect on the live site, so day-to-day work cannot break the demo and a half-finished migration cannot reach live data. To ship:

```
git checkout deploy
git merge main
git push origin deploy      # Render and Vercel both rebuild
git checkout main
```

The database is never at risk from a redeploy — it lives on Aiven, outside the container. The only thing lost on redeploy is uploaded profile photos, because the container filesystem is ephemeral.

7. What is unverified, and the free-tier limits

Read before you start, so nothing is a surprise

The Docker image has never been built. There was no Docker daemon on the machine it was written on, so **Render's first build log is its first real test.** Budget time for one or two build fixes. Section 10 of [docs/DEPLOYMENT.md](#) maps symptoms to causes; send the build log over if it fails.

Limit	What it means in practice
API sleeps after 15 minutes idle	The first request then takes 30–60 seconds. Open the site a minute before presenting, or point a free pinger (cron-job.org, UptimeRobot) at <code>/up</code> every 10 minutes.
No cron on the free tier	Weekly bundling, the nightly archive purge and hourly journal reminders do not run automatically. Admin → System Settings has manual triggers for bundling and the purge, which is what a demo needs.
Ephemeral filesystem	Uploaded avatars are discarded on redeploy. Setting <code>AVATAR_DISK=r2</code> plus the <code>R2_*</code> variables fixes it permanently — already wired, a pure environment change — but Cloudflare R2 requires a card.
<code>MAIL_MAILER=log</code>	No email is sent at all. Intentional: reminder email only targets Google-verified addresses, of which a freshly seeded database has none.

Verified before handover: 395 of 395 backend tests passing (including every PDF export), `npm run build` clean, `composer audit` clean after clearing 10 dependency advisories, and login confirmed end-to-end through the proxy. · **Also note:** self-service registration (`POST /register`) was removed — it had shipped since Phase 1 and created an active student account for any anonymous caller, which

only became dangerous once the app was publicly reachable.